

Etkinlik, Biletleme ve Koltuk Rezervasyon Sistemi

Konser, tiyatro, konferans ve benzeri etkinliklerin oluşturulabildiği; kullanıcıların etkinlikleri inceleyebildiği, koltuk seçerek rezervasyon ve bilet satın alma işlemleri gerçekleştirebildiği web tabanlı bir sistem geliştirilecektir.

- Backend: .NET Web API
- Frontend: React ve TypeScript

Bu proje yalnızca CRUD işlemlerinden oluşmamalıdır. Gerçek bir biletleme sisteminde karşılaşılan eş zamanlı rezervasyon, kapasite kontrolü, geçici koltuk kilitleme, ödeme sonucu işleme, bildirim, loglama, cache ve background job gibi problemler ele alınmalıdır.

Projenin Amacı

Bu projede stajyerlerin aşağıdaki konularda gelişmesi amaçlanmaktadır:

- İş analizi
- Domain modelleme
- Onion Architecture
- REST API tasarımı
- Authentication ve authorization
- Rol bazlı ve policy bazlı yetkilendirme
- Entity Framework Core
- PostgreSQL
- Transaction yönetimi
- Optimistic concurrency
- Distributed locking yaklaşımı
- Redis cache
- Background job
- Event-driven yaklaşım
- Gerçek zamanlı iletişim
- Dosya ve görsel yönetimi
- Ödeme sistemi simülasyonu
- API güvenliği
- Test yazımı
- Docker
- CI/CD
- Monitoring ve observability
- Frontend state yönetimi
- Responsive kullanıcı arayüzü
- Teknik dokümantasyon

- Git ile ekip çalışması

Kullanılacak Teknolojiler

Backend

- .NET 9
- ASP.NET Core Web API
- C#
- Entity Framework Core
- PostgreSQL
- Onion Architecture
- CQRS
- MediatR
- FluentValidation
- Mapster veya AutoMapper
- JWT Authentication
- Refresh Token
- Redis
- SignalR
- Hangfire veya Quartz.NET
- Serilog
- OpenTelemetry
- Swagger / OpenAPI
- xUnit
- FluentAssertions
- Moq veya NSubstitute
- Testcontainers
- Docker
- Docker Compose

Frontend

- React
- TypeScript
- Vite
- React Router
- TanStack Query
- Zustand veya Redux Toolkit
- Axios
- React Hook Form
- Zod
- Material UI, Ant Design veya Tailwind CSS
- SignalR Client
- Recharts

- Vitest
- React Testing Library
- Playwright veya Cypress

Diğer Araçlar

- Git
- GitHub veya GitLab
- Postman
- Swagger
- Docker Desktop
- pgAdmin
- Redis Insight
- Figma
- draw.io
- SonarQube
- GitHub Actions veya GitLab CI
- Jira, Trello veya GitHub Projects

Mimari Yaklaşım

Backend tarafında Onion Architecture uygulanmalıdır.

Katmanlar

Domain

- Entity sınıfları
- Value Object yapıları
- Enum tanımları
- Domain kuralları
- Domain event tanımları
- Repository abstraction gerekiyorsa yalnızca interface seviyesinde
- Framework bağımlılığı içermeyen çekirdek yapı

Application

- Use case tanımları
- Command ve Query sınıfları
- Handler sınıfları
- DTO modelleri
- Validation
- Interface tanımları
- Authorization kontrolleri
- Mapping

- Application service yapıları

Infrastructure

- Entity Framework Core
- PostgreSQL
- Redis
- E-posta servisi
- Dosya saklama servisi
- Background job
- Harici servis adaptörleri
- Repository implementasyonları gerekiyorsa bu katmanda
- Logging ve telemetry yapılandırmaları

Presentation / Web API

- Controller veya Minimal API endpointleri
- Authentication
- Authorization
- Middleware
- Exception handling
- Swagger
- API versioning
- Request ve response modelleri

Tests

- Unit test
- Integration test
- Architecture test
- End-to-end test

Kullanıcı Roller

Sistemde aşağıdaki roller bulunacaktır:

Kullanıcı

- Etkinlikleri listeleyebilir.
- Etkinlik detaylarını görüntüleyebilir.
- Koltuk seçebilir.
- Rezervasyon oluşturabilir.
- Ödeme simülasyonu gerçekleştirebilir.
- Satın aldığı biletleri görüntüleyebilir.
- Biletini iptal edebilir.
- Favori etkinliklerini yönetebilir.

- Etkinliklere yorum ve puan verebilir.
- Bildirimlerini görüntüleyebilir.

Organizatör

- Etkinlik oluşturabilir.
- Kendi etkinliklerini güncelleyebilir.
- Salon ve oturma planı seçebilir.
- Bilet kategorileri ve fiyatları tanımlayabilir.
- Etkinlik satış durumunu görüntüleyebilir.
- Etkinlik raporlarını inceleyebilir.
- Etkinliği yayına alabilir veya iptal edebilir.

Admin

- Tüm kullanıcıları yönetebilir.
- Organizatör başvurularını onaylayabilir.
- Tüm etkinlikleri görüntüleyebilir.
- Uygunsuz etkinlikleri pasif hâle getirebilir.
- Kategori, şehir ve salon yönetimi yapabilir.
- Sistem raporlarını görüntüleyebilir.
- Audit log kayıtlarını inceleyebilir.

Sprint 1 – İş Analizi ve Domain Tasarımı

Sistemin iş analizi, temel domain modeli ve veritabanı tasarımı hazırlanacaktır.

İş Analizi

- Kullanıcı hangi işlemleri yapabilir?
- Organizatör hangi işlemleri yapabilir?
- Admin hangi işlemleri yapabilir?
- Bir etkinliğin yaşam döngüsü nasıl ilerler?
- Koltuk rezervasyonu nasıl yapılır?
- Koltuk kaç dakika kilitli tutulmalıdır?
- Aynı koltuğu iki kullanıcı aynı anda seçerse ne olmalıdır?
- Ödeme başarısız olduğunda rezervasyon ne olmalıdır?
- Etkinlik iptal edildiğinde biletler ne olmalıdır?
- Bilet iade politikası nasıl uygulanmalıdır?
- Hangi işlemlerde transaction gerekir?
- Hangi işlemlerde cache kullanılmalıdır?
- Hangi işlemler background job ile yapılmalıdır?
- Hangi işlemler loglanmalıdır?
- Hangi senaryolarda kullanıcıya bildirim gönderilmelidir?

- Hangi alanlara index eklenmelidir?

Etkinlik Durumları

- Draft
- PendingApproval
- Published
- SalesOpen
- SalesClosed
- Completed
- Cancelled
- Suspended

Rezervasyon Durumları

- Pending
- Locked
- PaymentPending
- Confirmed
- Expired
- Cancelled
- Refunded

Ödeme Durumları

- Pending
- Processing
- Successful
- Failed
- Cancelled
- Refunded

Bilet Durumları

- Active
- Used
- Cancelled
- Refunded
- Expired

ER Diyagramı

- Users
- Roles

- UserRoles
- OrganizerProfiles
- OrganizerApplications
- Events
- EventCategories
- Cities
- Venues
- Halls
- SeatLayouts
- SeatSections
- Seats
- EventSessions
- TicketTypes
- EventSeats
- Reservations
- ReservationItems
- Payments
- PaymentTransactions
- Tickets
- TicketQrCodes
- Favorites
- Reviews
- Notifications
- RefreshTokens
- AuditLogs
- OutboxMessages
- UploadedFiles

Veritabanı Tasarımı

- Primary Key yapıları
- Foreign Key ilişkileri
- Composite Key kullanılan tablolar
- Unique alanlar
- Index gerektiren alanlar
- Nullable alanlar
- Cascade Delete davranışları
- Soft Delete kullanılacak tablolar
- CreatedAt ve UpdatedAt alanları
- Concurrency Token kullanılacak tablolar
- Audit alanları
- Para birimi ve ücret alanlarının veri tipi
- Tarih ve saat bilgilerinin UTC tutulması

Önemli Unique Kuralları

- Aynı etkinlik oturumunda aynı koltuk yalnızca bir kez bulunmalıdır.
- Bilet numarası benzersiz olmalıdır.
- QR kod değeri benzersiz olmalıdır.
- Aynı kullanıcı aynı etkinliği bir kez favorileyebilmelidir.
- Aynı kullanıcı aynı etkinliğe yalnızca bir yorum yapabilmelidir.

Sprint 2 – Proje Kurulumu, Onion Architecture ve Docker

Backend ve frontend projelerinin temel yapıları oluşturulacaktır.

Backend Solution Yapısı

Örnek:

- Ticketing.Domain
- Ticketing.Application
- Ticketing.Infrastructure
- Ticketing.Persistence
- Ticketing.WebApi
- Ticketing.UnitTests
- Ticketing.IntegrationTests
- Ticketing.ArchitectureTests

Backend Gereksinimleri

- Dependency Injection yapılandırılmalıdır.
- Katman bağımlılıkları doğru kurulmalıdır.
- Domain katmanı Infrastructure katmanını referans almamalıdır.
- Application katmanı Web API katmanını referans almamalıdır.
- Architecture testleri yazılmalıdır.
- Ortak result modeli hazırlanmalıdır.
- Pagination modeli hazırlanmalıdır.
- Global exception handling eklenmelidir.
- Problem Details standardı kullanılmalıdır.

Docker Compose

Docker Compose ile aşağıdaki servisler ayağa kaldırılmalıdır:

- PostgreSQL
- Redis
- Backend API

- Frontend
- pgAdmin
- Redis Insight
- Mailpit veya benzeri yerel e-posta test servisi

Environment Yapısı

Aşağıdaki bilgiler environment variable olarak yönetilmelidir:

- Database connection string
- Redis connection string
- JWT secret
- Access token süresi
- Refresh token süresi
- E-posta ayarları
- Dosya saklama ayarları
- Uygulama URL bilgileri

Sprint 3 – Authentication ve Authorization

Kimlik doğrulama ve yetkilendirme sistemi geliştirilecektir.

Backend Endpointleri

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh-token
- POST /api/v1/auth/revoke-token
- POST /api/v1/auth/logout
- GET /api/v1/auth/me
- POST /api/v1/auth/change-password
- POST /api/v1/auth/forgot-password
- POST /api/v1/auth/reset-password

Authentication Gereksinimleri

- Access Token oluşturulmalıdır.
- Refresh Token oluşturulmalıdır.
- Refresh Token rotation uygulanmalıdır.
- Eski Refresh Token tekrar kullanılamamalıdır.
- Logout işleminde token iptal edilmelidir.
- Şifreler güvenli biçimde hashlenmelidir.

- Şifre sıfırlama tokeni süreli olmalıdır.
- Hassas bilgiler loglanmamalıdır.

Authorization

- Role based authorization
- Policy based authorization
- Resource based authorization

uygulanmalıdır.

Örnek policyler:

- OrganizerOnly
- AdminOnly
- EventOwner
- TicketOwner
- ReservationOwner

Frontend Görevleri

- Login
- Register
- Şifremi unuttum
- Şifre sıfırlama
- Protected Route
- Rol bazlı route
- Axios interceptor
- Token yenileme
- Logout
- Yetkisiz erişim ekranı
- Session expired ekranı

Sprint 4 – Salon, Oturma Planı ve Koltuk Yönetimi

Admin tarafında salon ve koltuk yapısı geliştirilecektir.

Backend Modülleri

Venue

- Listeleme
- Oluşturma

- Güncelleme
- Silme
- Detay görüntüleme

Hall

- Listeleme
- Oluşturma
- Güncelleme
- Silme

Seat Layout

- Oturma planı oluşturma
- Bölüm oluşturma
- Sıra oluşturma
- Koltuk oluşturma
- Koltuk devre dışı bırakma

Endpoint Örnekleri

- POST /api/v1/venues
- GET /api/v1/venues
- POST /api/v1/venues/{venueId}/halls
- POST /api/v1/halls/{hallId}/seat-layouts
- POST /api/v1/seat-layouts/{id}/sections
- POST /api/v1/seat-layouts/{id}/generate-seats
- GET /api/v1/seat-layouts/{id}

İş Kuralları

- Aynı salonda aynı isimde iki oturma planı bulunmamalıdır.
- Aynı bölümde aynı sıra ve koltuk numarası tekrar edemez.
- Kullanılmış oturma planı fiziksel olarak silinmemelidir.
- Aktif etkinlik bulunan salon silinmemelidir.
- Koltuk kapasitesi salon kapasitesini aşmamalıdır.

Frontend Görevleri

- Mekân listeleme
- Salon yönetimi
- Oturma planı tasarlama ekranı
- Bölüm ekleme
- Sıra ve koltuk oluşturma
- Görsel koltuk planı
- Koltuk aktif/pasif durumu

- Plan önizleme

Sprint 5 – Etkinlik ve Oturum Yönetimi

Organizatörlerin etkinlik oluşturabilmesi sağlanacaktır.

Etkinlik Endpointleri

- POST /api/v1/events
- GET /api/v1/events
- GET /api/v1/events/{id}
- PUT /api/v1/events/{id}
- DELETE /api/v1/events/{id}
- POST /api/v1/events/{id}/publish
- POST /api/v1/events/{id}/cancel
- POST /api/v1/events/{id}/sessions

Etkinlik Alanları

- Title
- Description
- CategoryId
- OrganizerId
- CityId
- VenueId
- HallId
- PosterImage
- MinimumAge
- Duration
- SalesStartDate
- SalesEndDate
- EventDate
- Status
- CancellationPolicy

Oturum Alanları

- EventId
- StartDate
- EndDate
- HallId
- SeatLayoutId
- Status

İş Kuralları

- Bitiş tarihi başlangıç tarihinden önce olamaz.
- Satış başlangıç tarihi satış bitiş tarihinden sonra olamaz.
- Satış bitiş tarihi etkinlik başlangıcından sonra olamaz.
- Aynı salon aynı zaman aralığında iki etkinliğe atanamaz.
- Yayına alınmış etkinliğin kritik alanları kontrolsüz değiştirilemez.
- Satışı başlamış etkinliğin oturma planı değiştirilemez.
- Etkinlik iptal edildiğinde aktif rezervasyonlar işlenmelidir.

Frontend Görevleri

- Etkinlik oluşturma formu
- Etkinlik düzenleme
- Etkinlik listesi
- Etkinlik detay sayfası
- Oturum ekleme
- Görsel yükleme
- Önizleme
- Yayına alma
- İptal etme
- Form doğrulama

Sprint 6 – Bilet Türü ve Fiyatlandırma

Etkinlik bazlı bilet kategorileri ve fiyatlandırma sistemi hazırlanacaktır.

Bilet Türleri

Örnek:

- Standard
- Student
- VIP
- EarlyBird
- Balcony
- FrontStage

Backend Endpointleri

- POST /api/v1/events/{eventId}/ticket-types
- GET /api/v1/events/{eventId}/ticket-types
- PUT /api/v1/ticket-types/{id}
- DELETE /api/v1/ticket-types/{id}

- POST /api/v1/ticket-types/{id}/assign-section

İş Kuralları

- Fiyat sıfırdan küçük olamaz.
- Satış başlamış bilet türünün fiyatı değiştirilirse değişiklik loglanmalıdır.
- Kontenjan salon kapasitesini aşamaz.
- Aynı koltuk birden fazla aktif bilet türüne atanamaz.
- Bilet türü satış tarih aralığı dışında satın alınamaz.
- Öğrenci bileti için doğrulama alanı tasarlanmalıdır.

Para Yönetimi

- Para değerleri decimal olarak tutulmalıdır.
- Floating point kullanılmamalıdır.
- Currency alanı bulunmalıdır.
- Toplam tutar backend tarafında hesaplanmalıdır.
- Frontend tarafından gönderilen toplam tutara güvenilmemelidir.

Sprint 7 – Koltuk Kilitleme ve Rezervasyon Sistemi

Projenin en önemli teknik sprintidir.

Kullanıcı koltuk seçtiğinde koltuğun belirli süre boyunca başka kullanıcılar tarafından seçilmesi engellenmelidir.

Rezervasyon Endpointleri

- POST /api/v1/reservations
- GET /api/v1/reservations/{id}
- GET /api/v1/users/me/reservations
- POST /api/v1/reservations/{id}/cancel
- POST /api/v1/reservations/{id}/extend
- GET /api/v1/event-sessions/{id}/seat-availability

İş Kuralları

- Aynı koltuk aynı anda iki aktif rezervasyonda bulunamaz.
- Rezervasyon süresi örneğin 10 dakika olmalıdır.
- Süresi dolan rezervasyon otomatik olarak iptal edilmelidir.
- Süresi dolmuş rezervasyon üzerinden ödeme başlatılamaz.
- Bir kullanıcı aynı oturum için belirlenen maksimum bilet sayısını aşamaz.

- Rezervasyon toplam tutarı backend tarafından hesaplanmalıdır.
- Koltuk durumu rezervasyon öncesinde tekrar kontrol edilmelidir.
- İşlem transaction içinde gerçekleştirilmelidir.
- Aynı isteğin tekrar gönderilmesine karşı idempotency uygulanmalıdır.

Concurrency Gereksinimleri

Aşağıdaki yöntemlerden en az biri araştırılıp uygulanmalıdır:

- Optimistic concurrency
- PostgreSQL row-level locking
- Redis distributed lock
- Unique constraint ile yarış durumu engelleme

Stajyerler seçtikleri yöntemin avantajlarını ve dezavantajlarını yazılı olarak açıklamalıdır.

Redis Kullanımı

Redis aşağıdaki amaçlarla kullanılabilir:

- Geçici koltuk kilidi
- Rezervasyon süresi
- Koltuk uygunluk cache'i
- Rate limiting verisi
- Distributed lock

Frontend Görevleri

- Görsel koltuk seçimi
- Dolu koltuk
- Boş koltuk
- Seçili koltuk
- Geçici olarak kilitli koltuk
- Rezervasyon geri sayımı
- Toplam fiyat hesap görünümü
- Rezervasyon özeti
- Süre doldu uyarısı
- Çakışma hatası görünümü

Sprint 8 – Ödeme Simülasyonu ve Bilet Oluşturma

Gerçek ödeme sağlayıcısı kullanılmak zorunda değildir. Ancak ödeme sağlayıcısı entegrasyonuna benzer bir yapı kurulmalıdır.

Payment Provider Abstraction

Aşağıdaki gibi bir abstraction oluşturulmalıdır:

- IPaymentService
- CreatePayment
- VerifyPayment
- RefundPayment
- CancelPayment

En az iki implementasyon hazırlanabilir:

- MockPaymentProvider
- FailedPaymentProvider

Backend Endpointleri

- POST /api/v1/payments
- POST /api/v1/payments/{id}/complete
- POST /api/v1/payments/{id}/fail
- POST /api/v1/payments/{id}/refund
- GET /api/v1/payments/{id}

İş Kuralları

- Ödeme yalnızca aktif rezervasyon için başlatılabilir.
- Aynı rezervasyon için birden fazla başarılı ödeme oluşamaz.
- Ödeme başarılı olmadan bilet oluşturulamaz.
- Ödeme başarılı olduğunda rezervasyon Confirmed olmalıdır.
- Her rezervasyon kalemi için bilet oluşturulmalıdır.
- Ödeme başarısız olduğunda koltuklar serbest bırakılmalıdır.
- İade işlemi transaction içinde yönetilmelidir.
- Callback işlemleri idempotent olmalıdır.

Bilet Oluşturma

Başarılı ödeme sonrasında:

- Benzersiz bilet numarası
- Benzersiz QR kod
- Kullanıcı bilgisi
- Etkinlik bilgisi
- Koltuk bilgisi
- Bilet türü
- Fiyat
- Oluşturulma tarihi

üretilmelidir.

Transaction Kullanımı

Aşağıdaki işlemler tek bir transaction veya tutarlı bir süreç içerisinde çalışmalıdır:

- Ödeme başarılı kaydı
- Rezervasyon onayı
- Bilet oluşturma
- Koltukların satıldı olarak işaretlenmesi
- Notification oluşturma
- Outbox message oluşturma

Sprint 9 – Outbox Pattern ve Background Job

Dağıtık sistemlere giriş amacıyla Outbox Pattern uygulanmalıdır.

Outbox Kullanım Senaryoları

- Bilet satın alındı e-postası
- Etkinlik iptal bildirimi
- Rezervasyon süresi doldu bildirimi
- Ödeme başarı bildirimi
- QR bilet oluşturma işlemi
- Rapor hazırlama

Outbox Tablosu

En az aşağıdaki alanları içermelidir:

- Id
- Type
- Payload
- CreatedAt
- ProcessedAt
- RetryCount
- ErrorMessage

Background Job

Hangfire veya Quartz.NET ile:

- Süresi dolan rezervasyonları iptal etme
- Outbox mesajlarını işleme
- Başarısız mesajları yeniden deneme
- Yaklaşan etkinlik hatırlatması
- Günlük satış özeti oluşturma

işlemleri gerçekleştirilmelidir.

İş Kuralları

- Aynı Outbox kaydı iki kez işlenmemelidir.
- Başarısız işlem yeniden denenmelidir.
- Belirli deneme sayısından sonra hata kaydı oluşturulmalıdır.
- Job sonuçları loglanmalıdır.
- Job işlemleri kullanıcı isteğini gereksiz yere bekletmemelidir.

Sprint 10 – SignalR ile Gerçek Zamanlı Koltuk Güncelleme

Koltuk durumlarının kullanıcı ekranlarında gerçek zamanlı güncellenmesi sağlanacaktır.

SignalR Olayları

- SeatLocked
- SeatReleased
- SeatSold
- ReservationExpired
- EventCancelled

İş Kuralları

- Kullanıcı yalnızca görüntülediği etkinlik oturumunun grubuna katılmalıdır.
- Bir koltuk başka kullanıcı tarafından seçildiğinde ekran güncellenmelidir.
- Rezervasyon süresi dolduğunda koltuk serbest görünmelidir.
- Satılan koltuk yeniden seçilememelidir.
- SignalR bağlantısı kesildiğinde frontend yeniden bağlanmalıdır.

Frontend Görevleri

- SignalR bağlantısı
- Otomatik yeniden bağlanma
- Gerçek zamanlı koltuk güncelleme
- Bağlantı durumu göstergesi

- Kullanıcıya çakışma bildirimi
- Güncel koltuk listesini yeniden çekme

Sprint 11 – Arama, Filtreleme, Pagination ve Cache

Gerçek dünya listeleme ihtiyaçları uygulanacaktır.

Etkinlik Filtreleri

- Şehir
- Kategori
- Tarih
- Fiyat aralığı
- Mekân
- Organizatör
- Yaş sınırı
- Satış durumu

Pagination

Örnek:

GET /api/v1/events?pageNumber=1&pageSize=20

Search

Örnek:

GET /api/v1/events?search=konser

Sorting

Örnek:

GET /api/v1/events?sortBy=startDate&sortDirection=asc

API Yanıtı

- Items
- PageNumber
- PageSize

- TotalCount
- TotalPages
- HasPreviousPage
- HasNextPage

Redis Cache

Aşağıdaki veriler cache edilebilir:

- Popüler etkinlikler
- Etkinlik kategorileri
- Şehir listesi
- Etkinlik detayları
- Salon oturma planı

Cache Kuralları

- Cache key standardı oluşturulmalıdır.
- Cache expiration tanımlanmalıdır.
- Veri güncellendiğinde ilgili cache temizlenmelidir.
- Kullanıcıya özel hassas veriler ortak cache içinde tutulmamalıdır.
- Cache kapalı olduğunda sistem çalışmaya devam edebilmelidir.

Sprint 12 – Yorum, Puanlama ve Favori Sistemi

Review Endpointleri

- POST /api/v1/events/{eventId}/reviews
- GET /api/v1/events/{eventId}/reviews
- PUT /api/v1/reviews/{id}
- DELETE /api/v1/reviews/{id}

İş Kuralları

- Puan 1 ile 5 arasında olmalıdır.
- Kullanıcı etkinlik başına bir yorum oluşturabilir.
- Yalnızca etkinliğe geçerli bilet almış kullanıcı yorum yapabilir.
- Etkinlik tamamlanmadan yorum yapılamaz.
- Kullanıcı yalnızca kendi yorumunu düzenleyebilir.
- Admin uygunsuz yorumu kaldırabilir.

Favori Endpointleri

- POST /api/v1/events/{eventId}/favorite
- DELETE /api/v1/events/{eventId}/favorite
- GET /api/v1/users/me/favorites

Sprint 13 – Organizatör Paneli ve Raporlama

Organizatör Dashboard

- Toplam etkinlik sayısı
- Yayındaki etkinlikler
- Toplam bilet satışı
- Toplam gelir
- İade edilen biletler
- Doluluk oranı
- En çok satan bilet türü
- Günlük satış grafiği
- Etkinlik bazlı gelir
- Bölüm bazlı doluluk oranı

Admin Dashboard

- Toplam kullanıcı
- Toplam organizatör
- Toplam etkinlik
- Aktif satışlar
- Toplam işlem hacmi
- İptal edilen etkinlikler
- Başarısız ödeme oranı
- En popüler şehirler
- En popüler kategoriler
- Sistem hata sayısı

Rapor Endpointleri

- GET /api/v1/reports/sales-summary
- GET /api/v1/reports/event-occupancy
- GET /api/v1/reports/revenue-by-event
- GET /api/v1/reports/ticket-type-sales
- GET /api/v1/reports/payment-statuses
- POST /api/v1/reports/export

Export

Raporlar aşağıdaki formatlardan birinde dışa aktarılabilir:

- Excel
- CSV
- PDF

Rapor üretimi background job olarak çalıştırılmalı ve tamamlandığında kullanıcıya bildirim gönderilmelidir.

Sprint 14 – Notification ve E-posta Sistemi

Bildirim Oluşturulacak İşlemler

- Rezervasyon oluşturulduğunda
- Rezervasyon süresi dolmak üzereyken
- Ödeme başarılı olduğunda
- Ödeme başarısız olduğunda
- Bilet oluşturulduğunda
- Etkinlik tarihi yaklaştığında
- Etkinlik iptal edildiğinde
- İade tamamlandığında
- Rapor hazırlandığında

Notification Endpointleri

- GET /api/v1/notifications
- GET /api/v1/notifications/unread-count
- PATCH /api/v1/notifications/{id}/read
- PATCH /api/v1/notifications/read-all

E-posta Şablonları

- Hoş geldiniz
- Şifre sıfırlama
- Rezervasyon oluşturuldu
- Ödeme başarılı
- Bilet bilgileri
- Etkinlik hatırlatma
- Etkinlik iptali
- İade tamamlandı

Yerel geliştirme ortamında Mailpit kullanılabilir.

Sprint 15 – API Güvenliği

Zorunlu Güvenlik Önlemleri

- Rate Limiting
- CORS yapılandırması
- Request size limit
- File type kontrolü
- MIME type kontrolü
- Güvenli dosya adı
- SQL Injection risk kontrolü
- XSS risk kontrolü
- Hassas veri maskeleyme
- JWT güvenliği
- Refresh Token güvenliği
- Brute force koruması
- Idempotency-Key desteği
- Security headers
- Input validation

Rate Limiting Örnekleri

- Login endpointi
- Register endpointi
- Şifre sıfırlama endpointi
- Rezervasyon oluşturma endpointi
- Ödeme endpointi
- Search endpointi

Idempotency

Aşağıdaki işlemlerde uygulanmalıdır:

- Rezervasyon oluşturma
- Ödeme başlatma
- Ödeme callback
- İade başlatma

Sprint 16 – Logging, Monitoring ve Observability

Serilog

Aşağıdaki işlemler loglanmalıdır:

- Login
- Başarısız login
- Etkinlik oluşturma
- Etkinlik yayınlama
- Rezervasyon oluşturma
- Koltuk kilitleme
- Ödeme
- İade
- Background job
- Cache hatası
- SignalR bağlantı hatası
- Beklenmeyen exception

Correlation ID

Her HTTP isteği için Correlation ID üretilmelidir.

Bu değer:

- Response header
- Application log
- Exception log
- Background job log
- Outbox kaydı

içerisinde kullanılmalıdır.

OpenTelemetry

En az aşağıdakiler takip edilmelidir:

- HTTP request trace
- Database sorguları
- Redis işlemleri
- Background job işlemleri
- Harici servis çağrıları

Health Check

- Database health check
- Redis health check
- Background job health check

- Storage health check

Endpoint örnekleri:

- GET /health
- GET /health/ready
- GET /health/live

Sprint 17 – Testing

Unit Test

En az aşağıdaki use case veya servisler test edilmelidir:

- Login
- Refresh Token
- Event Create
- Event Publish
- Seat Lock
- Reservation Create
- Reservation Expire
- Payment Complete
- Ticket Generate
- Refund
- Review Create

Integration Test

En az aşağıdaki senaryolar test edilmelidir:

- Register ve Login
- Etkinlik oluşturma
- Etkinlik yayınlama
- Rezervasyon oluşturma
- Aynı koltuğu iki kullanıcının almaya çalışması
- Süresi dolmuş rezervasyonda ödeme
- Başarılı ödeme sonrası bilet oluşturma
- Başarısız ödeme sonrası koltuk serbest bırakma
- İade işlemi
- Yetkisiz erişim

Testcontainers

Integration test sırasında aşağıdaki gerçek servisler container olarak çalıştırılmalıdır:

- PostgreSQL
- Redis

Architecture Test

Aşağıdaki kurallar test edilmelidir:

- Domain, Infrastructure katmanını referans alamaz.
- Application, Web API katmanını referans alamaz.
- Handler sınıfları doğru namespace altında bulunmalıdır.
- Controller doğrudan DbContext kullanmamalıdır.
- Domain Entity doğrudan DTO döndürmemelidir.

Frontend Testleri

- Login formu
- Etkinlik filtreleme
- Koltuk seçimi
- Rezervasyon sayacı
- API hata ekranı
- Yetkisiz route
- Ödeme sonucu
- SignalR güncellemesi
- Responsive görünüm

End-to-End Test

Playwright veya Cypress ile:

- Kullanıcı kayıt olur.
- Giriş yapar.
- Etkinlik bulur.
- Koltuk seçer.
- Rezervasyon oluşturur.
- Ödeme yapar.
- Biletini görüntüler.

senaryosu otomatik test edilmelidir.

Sprint 18 – Swagger ve API Dokümantasyonu

Swagger üzerinde aşağıdakiler bulunmalıdır:

- Endpoint açıklamaları
- Request örnekleri
- Response örnekleri
- Validation hataları
- Authentication gereksinimleri
- Yetkili roller
- Pagination açıklaması
- Problem Details hata modeli
- Idempotency-Key açıklaması
- API version bilgisi

API Versioning

Aşağıdaki yapı kullanılmalıdır:

- /api/v1/events
- /api/v1/reservations
- /api/v1/payments

API version stratejisi dokümente edilmelidir.

OpenAPI Client

Frontend API modellerinin manuel tekrar yazılmasını azaltmak amacıyla Swagger dokümanından TypeScript client üretimi araştırılmalıdır.

Kullanılabilecek araçlar:

- NSwag
- OpenAPI Generator
- Orval

Sprint 19 – CI/CD ve Kod Kalitesi

Pipeline Aşamaları

- Restore
- Build
- Lint
- Unit Test
- Integration Test
- Architecture Test
- Frontend Test
- Docker Image Build
- Security Scan

- Artifact oluşturma

Kod Kalitesi

- ESLint
- Prettier
- .NET Analyzer
- StyleCop
- SonarQube
- Test coverage raporu

kullanılmalıdır.

Branch Koruması

- Main branch doğrudan değiştirilememelidir.
- Pull Request zorunlu olmalıdır.
- Testler başarısızsa merge yapılamamalıdır.
- En az bir code review alınmalıdır.

Backend Sorumlulukları

- Onion Architecture
- Domain modelleme
- Database tasarımı
- Entity Framework Core
- Migration yönetimi
- Authentication
- Authorization
- CQRS
- MediatR
- Validation
- Transaction
- Concurrency
- Redis
- SignalR
- Background job
- Outbox Pattern
- Payment abstraction
- Logging
- Monitoring
- Testing
- Swagger
- Docker
- API güvenliği

- CI/CD desteđi

Frontend Sorumlulukları

- React ve TypeScript yapısı
- Tasarım sistemi
- Responsive tasarım
- Etkinlik listeleme
- Etkinlik detay sayfası
- Görsel koltuk seçimi
- Rezervasyon geri sayımı
- Ödeme akışı
- Kullanıcı biletleri
- Organizatör paneli
- Admin paneli
- API entegrasyonu
- State yönetimi
- Cache yönetimi
- SignalR bağlantısı
- Form doğrulama
- Error handling
- Loading state
- Empty state
- Frontend testleri
- Accessibility
- Performans ölçümü

Frontend Teknik Gereksinimleri

- React componentleri TypeScript ile yazılmalıdır.
- **any** kullanımı mümkün olduğunca yasaklanmalıdır.
- API istekleri component içinde dađınık şekilde yazılmamalıdır.
- Ortak API client oluşturulmalıdır.
- Server state için TanStack Query kullanılmalıdır.
- Client state için Zustand veya Redux Toolkit kullanılabilir.
- Formlar React Hook Form ile yönetilmelidir.
- Zod ile doğrulama yapılmalıdır.
- Route bazlı code splitting uygulanmalıdır.
- Error Boundary kullanılmalıdır.
- Skeleton loading hazırlanmalıdır.
- Responsive tasarım uygulanmalıdır.
- Keyboard navigation desteklenmelidir.
- Kritik sayfalar Lighthouse ile analiz edilmelidir.

Zorunlu Teknik Kurallar

- Backend .NET ile geliştirilecektir.
- Frontend React ve TypeScript ile geliştirilecektir.
- PostgreSQL kullanılacaktır.
- Onion Architecture uygulanacaktır.
- CQRS ve MediatR kullanılacaktır.
- FluentValidation kullanılacaktır.
- Entity Framework Core kullanılacaktır.
- Controller içinde iş kuralı yazılmamalıdır.
- Controller doğrudan DbContext kullanmamalıdır.
- Endpointler doğrudan Entity döndürmemelidir.
- Request ve response modelleri ayrılmalıdır.
- Problem Details kullanılmalıdır.
- Global Exception Middleware hazırlanmalıdır.
- Redis kullanılmalıdır.
- Docker Compose hazırlanmalıdır.
- Swagger eksiksiz olmalıdır.
- API versioning uygulanmalıdır.
- Transaction gerektiren işlemler transaction içinde yapılmalıdır.
- Eş zamanlı rezervasyon problemi çözülmelidir.
- Idempotency uygulanmalıdır.
- Outbox Pattern kullanılmalıdır.
- En az bir background job bulunmalıdır.
- SignalR ile gerçek zamanlı güncelleme yapılmalıdır.
- Unit ve integration test yazılmalıdır.
- Testcontainers kullanılmalıdır.
- Architecture test yazılmalıdır.
- Serilog kullanılmalıdır.
- Correlation ID uygulanmalıdır.
- Health check endpointleri hazırlanmalıdır.
- Hassas bilgiler environment variable ile yönetilmelidir.
- Migration dosyaları Git'e eklenmelidir.
- Frontend loading, empty ve error state içermelidir.
- Tüm proje Docker Compose ile ayağa kaldırılabilmelidir.

Kullanılması Beklenen Öğretici Araçlar

Backend Ekibi

- Visual Studio veya Rider
- Visual Studio Debugger
- Breakpoint

- Watch
- Call Stack
- Immediate Window
- Swagger
- Postman
- pgAdmin
- Redis Insight
- Docker Desktop
- EF Core CLI
- Serilog
- OpenTelemetry
- Hangfire Dashboard
- SonarQube
- xUnit
- Testcontainers
- dotnet-trace
- dotnet-counters

Frontend Ekibi

- VS Code veya WebStorm
- React Developer Tools
- Browser Developer Tools
- Network Panel
- Performance Panel
- Lighthouse
- TanStack Query Devtools
- Redux DevTools
- Figma
- Vitest
- React Testing Library
- Playwright
- ESLint
- Prettier
- Storybook

Stajyerlerin yalnızca kod yazması değil; debug, profiling, test, network analizi, log inceleme ve performans ölçümü yapması beklenmektedir.

Yapay Zekâ Kullanım Kuralları

Yapay zekâ öğrenmeyi desteklemek amacıyla kullanılabilir. Ancak projenin doğrudan yapay zekâya yaptırılması kabul edilmeyecektir.

İzin Verilen Kullanımlar

- Kavram arařtırmak
- Hata mesajını anlamak
- Alternatif çözüm yaklaşımlarını öğrenmek
- Test senaryosu fikirleri almak
- Yazılan kod için code review istemek
- Dokümantasyon taslağı oluşturmak
- Güvenlik ve performans risklerini arařtırmak

Yasaklanan Kullanımlar

- Bir sprintin tamamını yapay zekâya yazdırmak
- Anlaşılmayan kodu projeye eklemek
- Yapay zekâ kodunu test etmeden kullanmak
- Tek komutla tam modül ürettilip teslim etmek
- Kodun nasıl çalıştığını açıklayamamak
- Sahte commit geçmiři oluşturmak
- Kod kaynağını gizlemek
- Tüm projeyi tek commit ile teslim etmek